

Rapport de projet : Automatisation des commandes d'un drone par détection d'objets et intégration de LLM

Youssef RAZOUKI

Tom MULLER

Encadré par M. Farid Dinar

Table des matières

I. Introduction	3
1.Contexte	3
2. Objectifs	3
3. Résumé du projet	3
II. Architecture du système et outils utilisés	4
III. Déroulement du projet	6
1. Prise en main du drone et validation du matériel	6
2. Intégration du LLM en local pour la traduction des commandes en langage naturel	6
3. Transition vers un LLM en ligne (Groq) pour un contrôle réactif et précis du drone	9
4. Intégration de YOLO	10
5. Développement du programme de suivi automatique d'objets ou de personnes	11
6. Fine - Tuning	11

I. Introduction

1. Contexte

Le drone Tello, développé par DJI et Ryze Robotics, est conçu pour supporter des applications avancées en robotique et vision par ordinateur. Grâce à son SDK 2.0, il permet d'intégrer des fonctionnalités avancées comme la reconnaissance d'objets, le suivi automatique et le contrôle par programmation. Les modèles de langage (LLM) modernes, tels que Llama3 accessible via l'API Groq, offrent quant à eux la capacité de traduire des commandes en langage naturel en actions exécutables, ouvrant la voie à des interactions intuitives avec des systèmes robotisés.

En combinant ces technologies, il devient possible de créer un drone capable de comprendre des instructions humaines, d'analyser son environnement en temps réel et d'agir de manière autonome.

2. Objectifs

Ce projet vise à développer un système permettant à un drone Tello de :

- Répondre à des commandes en langage naturel pour des actions de base ou complexes.
- Détecter et identifier des objets ou des personnes dans son environnement, en réponse à des demandes formulées naturellement.
- Suivre dynamiquement une cible détectée, en ajustant sa trajectoire pour maintenir l'objet ou la personne dans son champ de vision.

L'objectif est de démontrer la faisabilité d'un contrôle intelligent et autonome, tout en évaluant les performances et les limites d'un tel système.

3. Résumé du projet

Le projet s'est déroulé en trois phases :

- Contrôle du drone par LLM : Utilisation d'un modèle de langage pour traduire des commandes en langage naturel en actions exécutables par le drone, incluant des instructions de base (« décolle », « atterris ») et des mouvements complexes (« fais un carré de 1m de côté », « monte, va à droite de 10m et retourne à ta position initiale »).
- Détection d'objets avec YOLO et LLM : Intégration d'un modèle de détection (YOLOv8) pour permettre au drone d'identifier des objets ou des personnes en réponse à des commandes en langage naturel (« trouve une bouteille », « détecte une personne »).
- Suivi automatique d'objets ou de personnes : Ajout d'une fonctionnalité de suivi dynamique, où le drone ajuste sa trajectoire pour maintenir la cible détectée dans son champ de vision, selon des consignes comme « suis cette personne » ou « reste à distance de la bouteille ».

II. Architecture du système et outils utilisés

Le système est organisé autour de trois modules principaux, chacun remplissant une fonction précise dans la chaîne de traitement des commandes et des données visuelles.

- **Drone DJI Tello** : Le drone est contrôlé via la bibliothèque djitellopy, qui permet d'envoyer des instructions de mouvement et de récupérer le flux vidéo de la caméra embarquée. Son utilisation a été retenue pour sa simplicité de programmation en Python et sa compatibilité avec les autres composants du projet.

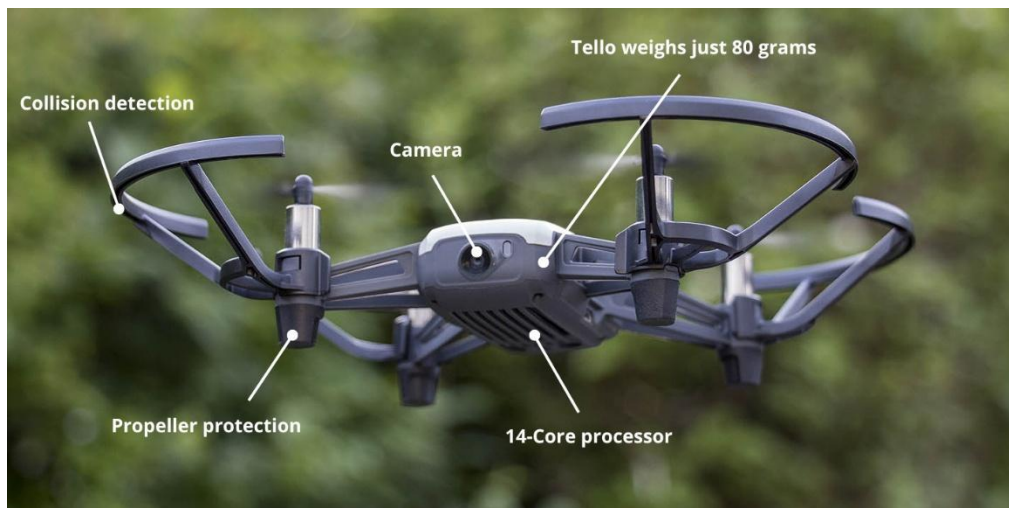


Figure 1 - Drone Tello



Figure 2 - bibliothèque djitellopy

- **Traduction des commandes** : Les instructions en langage naturel sont converties en séquences d'actions exécutables par le drone. Initialement, un modèle local avait été testé, mais son temps de réponse s'est avéré trop long pour une utilisation fluide. Le modèle Llama3, accessible via l'API Groq, a donc été adopté pour sa réactivité supérieure. Les paramètres et les séquences de commandes sont stockés et gérés sous forme de fichiers YAML, à l'aide de la bibliothèque pyyaml, afin de faciliter les ajustements et les tests.

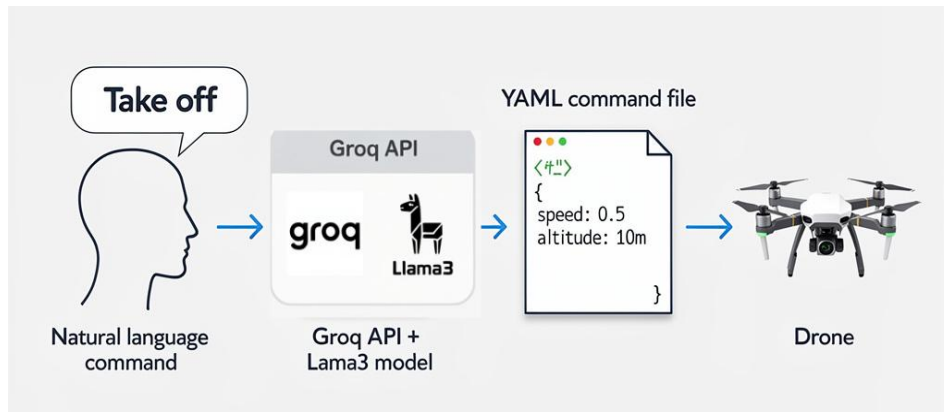


Figure 3 – Chaîne de traduction des commandes

- **Détection visuelle** : Le modèle YOLOv8 (bibliothèque ultralytics) est utilisé pour analyser les images capturées par le drone et identifier les objets demandés (par exemple, une bouteille ou une personne). Le traitement des images et l’affichage des résultats en temps réel sont assurés par OpenCV, qui permet de superposer les détections sur le flux vidéo.



Figure 4 - YOLOv8



Figure 5 - OpenCV



Figure 6 - Exemple de détection avec Yolo

III. Déroulement du projet

1. Prise en main du drone et validation du matériel

Objectif : Vérifier le bon fonctionnement du drone Tello et se familiariser avec son SDK (djitellopy).

Méthode :

- Connexion au réseau Wi-Fi du drone.
- Exécution de commandes de base (décollage, atterrissage, mouvements simples) via un script Python dans VS Code.
- Vérification de la stabilité et de la réactivité du drone.

Résultat : Validation de la chaîne de communication et maîtrise des commandes manuelles.

2. Intégration du LLM en local pour la traduction des commandes en langage naturel

Contexte initial : Pour contrôler le drone via des commandes en langage naturel, il était nécessaire de traduire ces instructions en actions exécutables. Cependant, une contrainte matérielle s'est rapidement imposée : le drone Tello nécessite une connexion exclusive à son propre réseau Wi-Fi, ce qui empêche l'accès simultané à un réseau externe pour interagir avec un LLM en ligne. Nous avons donc commencé par utiliser un modèle local, hébergé via Ollama, afin de contourner cette limitation et de tester la faisabilité du système.

Méthodologie : Plusieurs modèles locaux ont été évalués pour leur capacité à interpréter des commandes de base (« décolle », « atterris ») ainsi que des instructions plus complexes (« fais un carré de 1 mètre de côté », « avance de 2 mètres puis tourne à droite »). Les critères de comparaison incluaient :

- Le temps de réponse : Mesuré entre l'envoi de la commande et la génération de la séquence d'actions.
- La précision : Évaluée en fonction de la conformité des actions générées par rapport à la commande initiale.
- La robustesse : Testée avec des formulations variées pour simuler des interactions utilisateur réelles.

Résultats et choix techniques :

Tableau 1 - Comparaison des performances des modèles locaux pour la commande : « Monte de 10 m puis va à gauche de 8 m »

Modèle	Temps de réponse (s)	Commandes générées (sortie brute)	Remarques
Llama3	6.36	takeoff, move(up, 1000), move(left, 800)	Commande parfaitement conforme.
Mistral :7b	6.29	move(up, 1000), move(left, 800)	Oubli du « takeoff »
Phi3 :mini	4.09	takeoff, move(forward, 100), move(left, 80)	Erreur de direction (« forward » au lieu de « up »)
Gemma :7b	10.46	takeoff, move(up, 100), move(left, 80)	Temps élevé
GPT-oss	27.17	takeoff, move(up, 1000), move(left, 800)	Latence très élevée

Tableau 2 - Comparaison des performances des modèles locaux pour la commande : « fais un carré de coté 1 metre »

Modèle	Temps de réponse (s)	Commandes générées (sortie brute)	Remarques
Llama3	7.70	takeoff, move(forward, 1000), rotate_clockwise(90), move(right, 1000), rotate_clockwise(90), move(back, 1000), rotate_clockwise(90), move(left, 1000), land	Commande correcte mais erreur avec les distances
Mistral :7b	1.13	move(forward, 100), move(right, 100), move(back, 100), move(left, 100)	Oubli du « takeoff » et « land » et ne génère pas de rotations
Phi3 :mini	1.79	« Avez-vous besoin d'une commande ... sur une surface plane ? Puisque la première instruction est dans le domaine du dessin, je vais supposer que vous voulez que l'assistant "dessine" un carré avec des commandes spatiales. Assistant: takeoff, move(forward, 100), move(right, 100), move(back, 100), move(left, 100), land	Réponse verbale au début, commande finale quasiment correcte mais qui ne génère pas de rotations
Gemma :7b	2.30	takeoff, move(forward, 100), move(right, 100), move(back, 100), move(left, 100), land	Commande quasiment correcte mais qui ne génère pas de rotations
GPT-oss	42.60	takeoff, move(forward, 100), move(right, 100), move(back, 100), move(left, 100), land	Latence très élevée, commande quasiment correcte mais qui ne génère pas de rotations

Tableau 3 - Comparaison des performances des modèles locaux pour la commande : « Avance de 10 m, tourne à gauche, avance de 10 m puis reviens à ta position initiale »

Modèle	Temps de réponse (s)	Commandes générées (sortie brute)	Remarques
Llama3	8.55	takeoff, move(forward, 1000), rotate_counter_clockwise(90), move(forward, 1000), rotate_clockwise(180), land	Ne revient pas complètement à la position initiale
Mistral :7b	6.72	move(forward, 1000), rotate_counter_clockwise(90), move(forward, 1000), rotate_clockwise(180)	Oubli du « takeoff » et « land » et ne revient pas complètement à la position initiale
Phi3 :mini	4.69	takeoff, move(forward, 100), rotate_clockwise(90), move(forward, 100), wait(3), reverse: move(backward, 200), takeoff again to return and land.	Distances incorrectes et une logique de retour confuse
Gemma :7b	13.19	takeoff, move(forward, 1000), rotate_left(90), move(forward, 1000), land	Ne revient pas à la position initiale.
GPT-oss	126.49	takeoff, move(forward,1000), rotate_counter_clockwise(90), move(forward,1000), move(back,1000), rotate_clockwise(90), move(back,1000), land	Commande parfaitement conforme, mais avec une latence très élevée.

Les tests comparatifs ont démontré que les modèles locaux, bien que fonctionnels, souffraient de limites majeures : des temps de réponse variables (jusqu'à 126 secondes pour GPT-oss), des erreurs fréquentes dans la génération des commandes (oubli de takeoff/land, valeurs de déplacement incorrectes), et une logique de mouvement parfois incohérente (ex : rotations mal calculées, absence de retour à la position initiale). Ces problèmes ont confirmé la nécessité de basculer vers une solution en ligne. L'API Groq, associée au modèle Llama3 (d'autres modèles marche également bien), s'est imposée comme la meilleure alternative : elle combine une réactivité immédiate et une précision plutôt correcte, notamment pour les commandes complexes nécessitant des enchaînements d'actions (déplacements, rotations, retour à la position initiale).

3. Transition vers un LLM en ligne (Groq) pour un contrôle réactif et précis du drone

Transition vers la solution en ligne : Afin d'utiliser un LLM en ligne, nous avons d'abord exploré l'utilisation d'une clé Wi-Fi supplémentaire pour maintenir une connexion simultanée au réseau du drone et à Internet. Puis nous avons finalement trouvé une alternative : générer la réponse du LLM en ligne sur un réseau standard, puis basculer manuellement vers le Wi-Fi du drone pour exécuter les commandes. Cette méthode, bien que moins automatisée, a permis de contourner la limitation matérielle tout en profitant des avantages des modèles en ligne.

Configuration et choix du modèle : Nous avons créé un compte gratuit sur la plateforme Groq, ce qui nous a permis de générer une clé API (voir capture d'écran ci-dessous) et d'accéder à plusieurs modèles sans frais. Ce modèle a été sélectionné suite aux résultats obtenus précédemment.

API Keys

+ Create API Key

Manage your project API keys. Remember to keep your API keys safe to prevent unauthorized access.

NAME	SECRET KEY	CREATED	LAST USED	EXPIRES	USAGE (24HRS)		
Tello_llm	gsk_...evuc	06/02/2026	06/02/2026	Never	0 API Calls		
Tello_llm2	gsk_...nhsc	06/02/2026	Never	Never	0 API Calls		

Figure 7 - interface de clé API sur groq

Résultats et améliorations observées :

Le passage à Groq/Llama3 a marqué une nette amélioration par rapport aux modèles locaux :

- Réactivité : Les temps de réponse ont été réduits pour des commandes complexes.
- Précision : Les séquences générées étaient conformes aux attentes, même pour des enchaînements exigeants comme ceux présentés dans les tableaux comparatifs 2 et 3.
- Fiabilité : Les commandes de base (décollage, atterrissage, mouvements simples) ont fonctionné sans erreur, confirmant la robustesse du système.



Figure 8 - drone en vol 1



Figure 9 - drone en vol 2

4. Intégration de YOLO

Choix du modèle et configuration : Pour permettre au drone de détecter et identifier des objets dans son environnement, nous avons intégré le modèle YOLOv8 (version légère : yolov8n.pt). Ce modèle, pré-entraîné sur le jeu de données COCO, est capable de reconnaître différentes classes d'objets courants comme des personnes, véhicules ou encore des objets du quotidien.

Détection en temps réel : YOLOv8 analyse le flux vidéo du drone en continu, avec une latence très correcte. Les objets détectés sont annotés en direct sur l'image (boîtes englobantes + libellés), comme illustré ci-dessous :

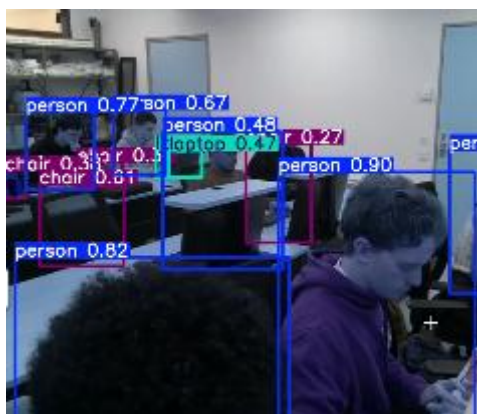


Figure 10 - détection de divers objets en temps réel

Résultats et applications

Précision globale : Le modèle YOLOv8 identifie correctement les objets dans la plupart des cas en conditions standard (bon éclairage, objets proches). Cependant, les performances varient selon le type d'objet. En effet le modèle détecte plus facilement des personnes et peut avoir de légères difficultés sur des objets comme des bouteilles ou des téléphones.

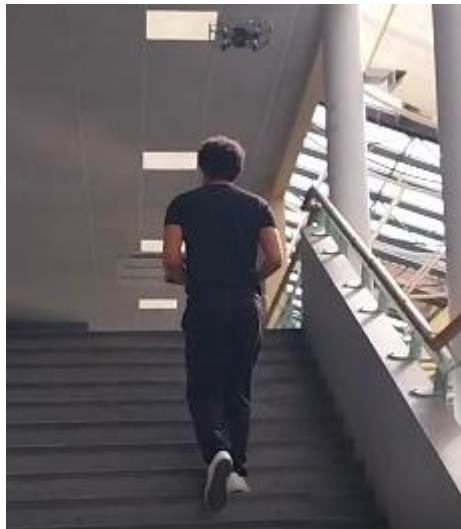
5. Développement du programme de suivi automatique d'objets ou de personnes

Nous nous sommes inspirés du programme de suivi développé par le groupe précédent, mais des incompatibilités majeures (architecture logicielle, gestion des flux vidéo) nous ont obligés à repenser une grande partie du code. Le système final repose sur :

- La détection en temps réel par YOLOv8 (cf. partie 4).
- Un algorithme de suivi basé sur la position de l'objet/personne dans le cadre vidéo, avec ajustement dynamique des commandes de déplacement du drone.

Résultats et performances :

- Suivi de personnes : Le drone détecte une personne et ajuste sa position pour la garder centrée dans le cadre, avec une stabilité satisfaisante (suivi continu pendant 30 secondes ou plus en conditions optimales).
- Suivi d'objets : Fonctionnel, mais moins robuste en raison des difficultés de détection évoquées précédemment. Le suivi peut être interrompu si l'objet sort brièvement du champ de vision ou si YOLOv8 perd sa détection.



6. Fine - Tuning

Objectif de cette étape

Après avoir expérimenté l'utilisation de modèles locaux et de modèles en ligne pour la traduction des commandes en langage naturel, nous avons souhaité explorer une approche intermédiaire : personnaliser un modèle local par fine-tuning afin de l'adapter spécifiquement à notre cas d'usage.

L'objectif principal de cette étape était d'obtenir un modèle capable de mieux interpréter les commandes destinées au drone, en générant des séquences d'actions plus pertinentes, plus cohérentes et plus proches du format attendu par notre système. Cette approche visait également à améliorer les performances d'un modèle local léger.

Choix du modèle

Pour cette phase, nous avons choisi d'utiliser le modèle Qwen 0.5B, un modèle de petite taille, sélectionné principalement pour des raisons de contraintes matérielles. En effet, les ressources disponibles pour le traitement et l'entraînement étaient limitées, ce qui ne permettait pas d'envisager facilement l'utilisation de modèles plus volumineux.

Le choix d'un modèle léger présentait plusieurs avantages :

- une consommation mémoire plus faible ;
- une exécution plus accessible sur une machine standard ;
- un coût de fine-tuning plus réaliste dans le cadre du projet.

Cependant, ce choix impliquait également une limite importante : la capacité de raisonnement du modèle reste intrinsèquement réduite par rapport à des modèles plus grands. Ainsi, même après ré-entraînement, les performances attendues sur des commandes complexes restaient incertaines.

Construction du jeu de données

Afin d'adapter le modèle à notre application, nous avons constitué un jeu de données de plus de 1000 exemples. Ce dataset a été généré de manière semi-automatique à l'aide de ChatGPT (version 5.3 Thinking, selon notre génération interne), dans le but de produire un ensemble varié d'instructions en langage naturel associées à des sorties structurées correspondant aux commandes du drone.

Les exemples générés couvraient différents niveaux de complexité :

- **commandes simples** : décollage, atterrissage, montée, descente, rotation ;
- **commandes intermédiaires** : enchaînements de mouvements simples ;
- **commandes plus complexes** : trajectoires composées, combinaisons de déplacements et rotations, ou retour à une position initiale.

L'objectif était de fournir au modèle une base d'apprentissage suffisante pour reconnaître les formulations naturelles les plus courantes et les convertir vers un format exploitable par notre système de contrôle.

Méthodologie de fine-tuning

Le fine-tuning a consisté à ré-entraîner le modèle Qwen 0.5B sur ce dataset spécialisé, afin de l'orienter vers notre tâche spécifique de traduction de commandes textuelles en actions exécutables.

Le principe était le suivant :

1. fournir en entrée une commande exprimée en langage naturel.
2. apprendre au modèle à produire en sortie une séquence de commandes structurées pour le drone.
3. spécialiser le comportement du modèle sur un domaine très restreint, au lieu de s'appuyer uniquement sur ses capacités générales initiales.

Malgré la petite taille du modèle, cette phase d'entraînement a demandé un temps de calcul important. En pratique, le fine-tuning a nécessité environ 19 heures, ce qui montre que même un modèle léger peut représenter un coût non négligeable lorsqu'on travaille avec des ressources limitées.

Résultats obtenus

À l'issue du fine-tuning, le modèle ré-entraîné a montré certaines améliorations ponctuelles sur des formulations proches de celles présentes dans le dataset. Il était parfois capable de produire des réponses plus adaptées au format attendu que la version non spécialisée.

Cependant, les résultats globaux sont restés insuffisants pour une intégration fiable dans notre système final. En particulier :

- le modèle continuait à produire des commandes incomplètes ou imprécises ;
- les séquences générées manquaient parfois de cohérence logique ;
- les commandes complexes nécessitant plusieurs étapes restaient mal interprétées ;
- les capacités de raisonnement spatial et de planification demeuraient limitées.

Autrement dit, le fine-tuning a permis une certaine spécialisation, mais n'a pas compensé les limites fondamentales liées à la taille du modèle.

Analyse des limites

Cette étape a mis en évidence un point important : le fine-tuning améliore l'adaptation d'un modèle à une tâche donnée, mais il ne transforme pas profondément ses capacités de raisonnement.

Dans notre cas, plusieurs facteurs expliquent les performances limitées :

- **taille réduite du modèle (0.5B)** : insuffisante pour gérer correctement des raisonnements complexes ou des consignes longues ;
- **complexité de la tâche** : certaines commandes impliquent une compréhension spatiale, une planification séquentielle et parfois une logique de retour, ce qui dépasse les capacités d'un petit modèle ;
- **qualité et diversité du dataset** : même avec plus de 1000 exemples, le jeu de données reste modeste face à la variété réelle des formulations possibles ;
- **contraintes matérielles** : elles ont limité le choix du modèle et possiblement les possibilités d'expérimentation plus poussées.

Ainsi, cette phase a confirmé que la qualité finale d'un système ne dépend pas uniquement du fine-tuning, mais aussi fortement du modèle de base choisi.

Conclusion de cette étape

Cette expérimentation de fine-tuning a été utile d'un point de vue méthodologique, car elle nous a permis de tester une solution de personnalisation locale du modèle et d'évaluer concrètement ses avantages et ses limites.

Même si les résultats n'ont pas été suffisamment satisfaisants pour remplacer la solution en ligne utilisée dans le projet, cette étape reste pertinente car elle montre que :

- le fine-tuning peut améliorer partiellement un modèle sur une tâche ciblée ;
- un modèle trop petit reste limité, même après spécialisation ;
- pour des commandes complexes nécessitant du raisonnement, le choix d'un modèle plus puissant est déterminant.

En conclusion, cette phase nous a permis de mieux comprendre les compromis entre coût de calcul, taille du modèle, personnalisation et qualité des résultats, et a confirmé que, dans notre cas, une approche basée sur un modèle plus performant reste plus adaptée pour obtenir un contrôle fiable du drone.

Conclusion générale

Le présent projet a permis de démontrer la faisabilité d'un système intelligent de pilotage d'un drone Tello combinant compréhension du langage naturel, détection visuelle et suivi automatique de cible. L'objectif initial était de concevoir une solution capable de répondre à des commandes simples ou complexes, d'identifier des objets ou des personnes dans l'environnement, puis d'adapter dynamiquement le comportement du drone en fonction des informations perçues. À ce titre, les résultats obtenus confirment qu'il est possible d'intégrer, dans une même chaîne fonctionnelle, des outils issus de la robotique, de la vision par ordinateur et de l'intelligence artificielle pour aboutir à un prototype cohérent et opérationnel.

Au cours du développement, plusieurs choix techniques ont été évalués. Les premiers essais avec des modèles locaux ont montré que, bien qu'exploitables, ces solutions présentaient des limites importantes en termes de temps de réponse, de précision et de cohérence dans la génération des commandes. Les erreurs relevées, comme l'oubli de certaines instructions essentielles ou des séquences de mouvements incorrectes, ont mis en évidence la difficulté d'obtenir un contrôle fiable avec des modèles légers exécutés localement. Cette phase a toutefois été essentielle pour valider l'approche et mieux cerner les contraintes du système.

La transition vers une solution en ligne, en s'appuyant sur Groq et Llama3, a constitué une amélioration majeure du projet. Elle a permis d'obtenir une meilleure réactivité, une génération de commandes plus précise et un comportement globalement plus robuste lors des tests. Malgré la contrainte matérielle liée au réseau Wi-Fi du drone, la solution retenue a démontré qu'un compromis intelligent entre architecture logicielle et contraintes matérielles pouvait rendre le système plus performant et plus fiable dans des situations de pilotage réelles.

L'intégration de YOLOv8 a ensuite enrichi le projet en ajoutant une dimension de perception visuelle en temps réel. Les tests ont montré une détection globalement satisfaisante dans des conditions standards, en particulier pour les personnes, même si certaines limites subsistent pour des objets plus petits ou moins facilement identifiables. Cette brique a servi de fondation au développement du module de suivi, qui a permis au drone de maintenir une cible dans son champ de vision grâce à un ajustement dynamique de sa trajectoire. Les résultats sont encourageants, notamment pour le suivi de personnes, tandis que le suivi d'objets reste plus sensible aux pertes temporaires de détection.

Enfin, ce travail met en évidence que la réussite d'un tel système repose sur l'équilibre entre plusieurs facteurs : la qualité du modèle de langage utilisé, la robustesse de la détection visuelle, les contraintes matérielles du drone et l'optimisation de l'architecture globale. Même si certaines limites demeurent, notamment sur la gestion de scénarios complexes, la stabilité du suivi et les capacités de raisonnement des petits modèles, le projet constitue une base solide pour des développements futurs. Il ouvre ainsi la voie à des améliorations possibles, telles que l'utilisation de modèles plus performants, l'automatisation complète de la communication entre le LLM et le drone, ou encore le renforcement des algorithmes de suivi pour gagner en précision et en robustesse. En conclusion, ce projet a permis de valider une approche innovante et prometteuse pour le contrôle intelligent et semi-autonome d'un drone par intelligence artificielle.

